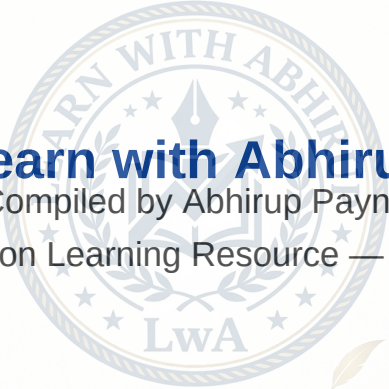


Python Programming Notes

The Complete A to Z Guide — From Absolute Zero to Advanced Mastery

The logo is a circular seal with a blue border. Inside the border, the words "LEARN WITH ABHIRUP" are written in a circular path at the top, and "LwA" is at the bottom. The center of the seal features a blue quill pen pointing upwards, flanked by two blue stars. Below the seal, the text "LEARN WITH ABHIRUP" is written in a bold, blue, sans-serif font, with a small yellow feather icon to the right.

Learn with Abhirup

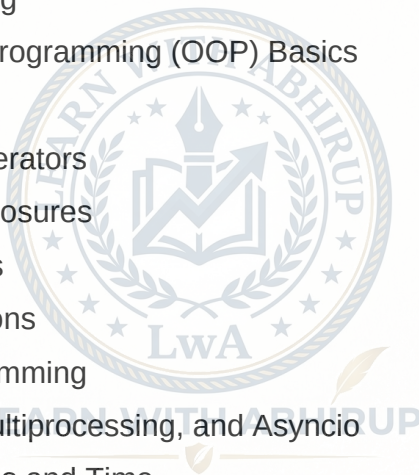
Compiled by Abhirup Payne

The Ultimate Python Learning Resource — Zero to Advanced

LEARN WITH ABHIRUP

Table of Contents

- Chapter 1: Introduction to Python
- Chapter 2: Variables and Data Types
- Chapter 3: Operators
- Chapter 4: Strings
- Chapter 5: Control Flow
- Chapter 6: Data Structures — Lists
- Chapter 7: Data Structures — Tuples, Sets, Dictionaries
- Chapter 8: Functions
- Chapter 9: Modules and Packages
- Chapter 10: File Handling
- Chapter 11: Exception Handling
- Chapter 12: Object-Oriented Programming (OOP) Basics
- Chapter 13: Advanced OOP
- Chapter 14: Iterators and Generators
- Chapter 15: Decorators and Closures
- Chapter 16: Context Managers
- Chapter 17: Regular Expressions
- Chapter 18: Functional Programming
- Chapter 19: Multithreading, Multiprocessing, and Asyncio
- Chapter 20: Working with Dates and Time
- Chapter 21: Popular Third-Party Libraries (Overview)
- Chapter 22: Testing in Python
- Chapter 23: Best Practices and PEP 8
- Chapter 24: Advanced Topics



Chapter 1: Introduction to Python

1.1 What is Python?

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. It emphasizes code readability with its clean, indentation-based syntax. Python is used in web development, data science, machine learning, automation, scripting, game development, and much more.

Key characteristics of Python:

- Interpreted: Code runs line by line through an interpreter, no separate compilation step needed.
- Dynamically typed: You do not declare variable types; Python figures it out at runtime.
- High-level: It abstracts away low-level details like memory management.
- Multi-paradigm: Supports procedural, object-oriented, and functional programming styles.
- Huge standard library and ecosystem: "batteries included" philosophy plus a massive third-party package index (PyPI).

1.2 Installing Python

Step by step:

1. Go to python.org/downloads and download the latest stable version (3.12 or newer recommended).
2. Run the installer. On Windows, check "Add Python to PATH" before clicking Install.
3. Verify installation by opening a terminal and typing:

```
python --version
```

or on some systems:

```
python3 --version
```

4. Install a code editor such as VS Code or PyCharm for a better development experience.

1.3 Running Python Code

There are three common ways to run Python:

1. Interactive shell (REPL): type `python` in your terminal, then type statements one at a time.
2. Script file: write code in a file ending with `.py`, then run `python filename.py`.
3. Jupyter Notebook: cell-based execution, popular for data science.

1.4 Your First Program

Every programmer's first program prints a greeting to the screen.

```
print("Hello, World!")
```

Step by step breakdown:

- `print` is a built-in function that outputs text to the console.
- The text inside quotes `"Hello, World!"` is a string literal.
- Parentheses `()` are used to call the function and pass arguments to it.

Run it by saving as `hello.py` and executing `python hello.py` in your terminal.

1.5 Comments

Comments are ignored by the interpreter and used to explain code.

```
# This is a single-line comment
print("This line runs") # Comments can also follow code

"""
This is a multi-line string,
often used as a comment block or docstring.
"""
```



Chapter 2: Variables and Data Types

2.1 Variables

A variable is a name that refers to a value stored in memory. Python variables do not need explicit type declarations.

```
name = "Abhirup"
age = 25
height = 5.9
is_student = True
```

Step by step:

- `name = "Abhirup"` creates a variable `name` and binds it to the string `"Abhirup"`.
- The `=` sign is the assignment operator, not mathematical equality.
- Python infers the type from the value on the right-hand side.

Rules for naming variables:

- Must start with a letter or underscore, not a digit.
- Can contain letters, digits, and underscores.
- Case-sensitive (`age` and `Age` are different).
- Cannot be a reserved keyword (like `if`, `for`, `class`).

2.2 Basic Data Types

Python has several built-in data types:

- `int` — whole numbers: `10`, `-5`, `1000`
- `float` — decimal numbers: `3.14`, `-0.001`
- `str` — text: `"hello"`, `'world'`
- `bool` — True or False
- `complex` — complex numbers: `3+4j`
- `NoneType` — represents absence of a value: `None`

```
x = 10          # int
y = 3.14        # float
name = "Python" # str
flag = True     # bool
z = 3 + 4j      # complex
nothing = None  # NoneType
```

Check a variable's type using `type()`:

```
print(type(x))    # <class 'int'>
print(type(name)) # <class 'str'>
```

2.3 Type Conversion (Casting)

You can convert between types explicitly.

```
a = "10"
b = int(a)      # string to int -> 10
c = float(b)    # int to float -> 10.0
d = str(c)      # float to string -> "10.0"
e = bool(0)     # 0 -> False
f = bool(5)     # nonzero -> True
```

Step by step:

- `int(a)` parses the string `"10"` and converts it to the integer `10`. This fails with a `ValueError` if the string is not a valid number.
- `bool()` treats `0`, `0.0`, `""`, `None`, and empty collections as `False`; everything else is `True`.

2.4 Dynamic Typing

A variable in Python can be reassigned to a different type at any time.

```
value = 5       # int
value = "five"  # now a string
value = [1, 2]  # now a list
```

This flexibility is powerful but can lead to bugs if not managed carefully; naming variables clearly helps avoid confusion.

Chapter 3: Operators

3.1 Arithmetic Operators

```
a, b = 10, 3
print(a + b)    # 13  addition
print(a - b)    # 7   subtraction
print(a * b)    # 30  multiplication
print(a / b)    # 3.333... true division (always returns float)
print(a // b)   # 3   floor division (rounds down to nearest integer)
print(a % b)    # 1   modulus (remainder)
print(a ** b)   # 1000 exponentiation (a to the power of b)
```

3.2 Comparison Operators

Return **True** or **False**.

```
print(5 == 5)   # True, equal to
print(5 != 3)   # True, not equal to
print(5 > 3)    # True, greater than
print(5 < 3)    # False, less than
print(5 >= 5)   # True, greater than or equal
print(5 <= 4)   # False, less than or equal
```

3.3 Logical Operators

```
x = True
y = False
print(x and y)  # False, both must be True
print(x or y)   # True, at least one must be True
print(not x)    # False, inverts the boolean
```

3.4 Assignment Operators

```
n = 10
n += 5    # n = n + 5  -> 15
n -= 3    # n = n - 3  -> 12
n *= 2    # n = n * 2  -> 24
n /= 4    # n = n / 4   -> 6.0
n //= 2   # n = n // 2  -> 3.0
n **= 2   # n = n ** 2  -> 9.0
n %= 4    # n = n % 4   -> 1.0
```

3.5 Identity and Membership Operators

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a is b)      # True, same object in memory
print(a is c)      # False, different objects, even if equal
print(a == c)      # True, same values
print(2 in a)       # True, membership check
print(5 not in a)   # True
```

Step by step:

- **is** checks object identity (same memory location), while **==** checks value equality.
- **in** and **not in** check whether a value exists inside a sequence like a list, string, or dictionary.



Chapter 4: Strings

4.1 Creating Strings

```
s1 = "Hello"
s2 = 'World'
s3 = """Triple quotes allow
multi-line strings"""
```

4.2 String Indexing and Slicing

Strings are sequences; each character has an index starting at 0.

```
s = "Python"
print(s[0])      # 'P' first character
print(s[-1])     # 'n' last character
print(s[0:3])    # 'Pyt' characters from index 0 up to (not including) 3
print(s[:2])     # 'Py' from the start up to index 2
print(s[2:])     # 'thon' from index 2 to the end
print(s[::-1])   # 'nohtyP' reversed string using step -1
```

4.3 String Methods

```
s = " Hello World "
print(s.strip())      # "Hello World" removes leading/trailing whitespace
print(s.lower())      # " hello world "
print(s.upper())      # " HELLO WORLD "
print(s.replace("World", "Python")) # replaces substring
print(s.split())      # ['Hello', 'World'] splits on whitespace by default
print("-".join(["a","b","c"])) # "a-b-c" joins list into a string
print(s.strip().startswith("Hello")) # True
print(s.strip().endswith("World"))   # True
print(len(s))         # length of the string including spaces
print(s.find("World")) # index where substring starts, -1 if not found
```

4.4 String Formatting

Three main ways to format strings:

```
name = "Abhirup"
age = 25

# 1. f-strings (recommended, Python 3.6+)
print(f"My name is {name} and I am {age} years old.")

# 2. .format() method
print("My name is {} and I am {} years old.".format(name, age))

# 3. % operator (old style)
print("My name is %s and I am %d years old." % (name, age))
```

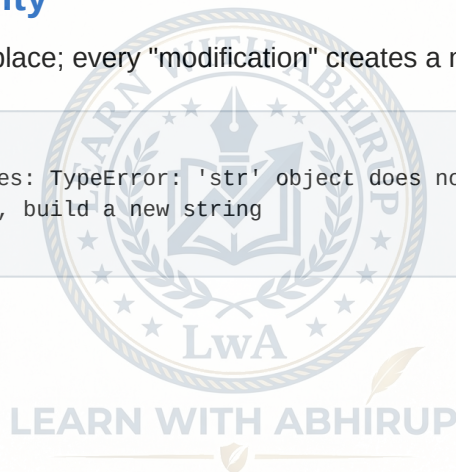
f-strings also support expressions and formatting specifiers:

```
pi = 3.14159
print(f"Pi rounded: {pi:.2f}") # "Pi rounded: 3.14"
print(f"{name.upper()}")      # calls a method inside the f-string
```

4.5 String Immutability

Strings cannot be changed in place; every "modification" creates a new string.

```
s = "hello"
# s[0] = "H" # This raises: TypeError: 'str' object does not support item assignment
s = "H" + s[1:] # Instead, build a new string
print(s) # "Hello"
```



Chapter 5: Control Flow

5.1 if, elif, else

```
age = 20
if age < 13:
    print("Child")
elif age < 20:
    print("Teenager")
else:
    print("Adult")
```

Step by step:

- Python checks conditions top to bottom.
- The first **True** condition's block executes; the rest are skipped.
- Indentation (commonly 4 spaces) defines the block, not braces like other languages.

5.2 Ternary (Conditional) Expression

```
age = 18
status = "Adult" if age >= 18 else "Minor"
print(status) # "Adult"
```

5.3 while Loops

```
count = 0
while count < 5:
    print(count)
    count += 1
```

This prints 0 through 4. Be careful to update the loop variable, or you get an infinite loop.

break and **continue**:

```
count = 0
while True:
    if count == 3:
        break # exits the loop entirely
    count += 1
    if count == 2:
        continue # skips the rest of this iteration
    print(count)
```

5.4 for Loops

```
for i in range(5):          # 0,1,2,3,4
    print(i)

fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

for index, fruit in enumerate(fruits):
    print(index, fruit)    # gives both position and value
```

range(start, stop, step):

```
for i in range(2, 10, 2):  # 2,4,6,8
    print(i)
```

5.5 Nested Loops and else on Loops

```
for i in range(3):
    for j in range(2):
        print(i, j)

for i in range(5):
    if i == 10:
        break
else:
    print("Loop completed without break") # this runs since break never triggered
```



Chapter 6: Data Structures — Lists

6.1 Creating and Accessing Lists

Lists are ordered, mutable collections.

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])      # "apple"
print(fruits[-1])     # "cherry"
print(fruits[1:3])    # ["banana", "cherry"]
```

6.2 Modifying Lists

```
fruits.append("mango")      # adds to the end
fruits.insert(1, "orange")  # inserts at index 1
fruits.remove("banana")     # removes first matching value
popped = fruits.pop()       # removes and returns the last item
fruits[0] = "pineapple"     # replace item at index 0
fruits.extend(["kiwi", "grape"]) # adds multiple items
fruits.sort()               # sorts in place, alphabetically/numerically
fruits.reverse()            # reverses order in place
print(len(fruits))          # number of items
```

6.3 List Comprehensions

A concise way to build lists.

```
squares = [x**2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

evens = [x for x in range(20) if x % 2 == 0]
# only even numbers from 0 to 19

pairs = [(x, y) for x in range(3) for y in range(2)]
# all combinations of x in range(3) and y in range(2)
```

Step by step for `[x**2 for x in range(10)]`:

- `for x in range(10)` iterates x from 0 to 9.
- `x**2` transforms each value.
- The whole expression collects results into a new list.

6.4 Common List Operations

```
nums = [5, 3, 8, 1, 9]
print(max(nums))    # 9
print(min(nums))    # 1
print(sum(nums))    # 26
print(sorted(nums)) # [1, 3, 5, 8, 9] returns new sorted list, original unchanged
print(nums.count(3)) # how many times 3 appears
print(nums.index(8)) # position of the first 8
```



Chapter 7: Data Structures — Tuples, Sets, Dictionaries

7.1 Tuples

Tuples are ordered and immutable (cannot be changed after creation).

```
point = (3, 4)
x, y = point          # unpacking
print(point[0])       # 3
# point[0] = 5        # Error: tuples don't support item assignment
single = (5,)         # a one-element tuple needs a trailing comma
```

Use tuples when data shouldn't change, such as coordinates or fixed records. They are also slightly faster and can be used as dictionary keys, unlike lists.

7.2 Sets

Sets are unordered collections of unique items.

```
s = {1, 2, 3, 3, 2}
print(s)              # {1, 2, 3} duplicates automatically removed

s.add(4)
s.remove(1)
print(3 in s)         # membership check, very fast

a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)          # union {1,2,3,4}
print(a & b)          # intersection {2,3}
print(a - b)          # difference {1}
print(a ^ b)          # symmetric difference {1,4}
```

7.3 Dictionaries

Dictionaries store key-value pairs.

```

person = {"name": "Abhirup", "age": 25, "city": "Kolkata"}
print(person["name"])          # "Abhirup"
print(person.get("email"))      # None, no KeyError since we used .get()
print(person.get("email", "N/A")) # "N/A" default value

person["email"] = "abhirup@example.com" # add new key
person["age"] = 26                       # update existing key
del person["city"]                       # remove a key

for key, value in person.items():
    print(key, value)

print(person.keys())    # dict_keys(['name', 'age', 'email'])
print(person.values())  # dict_values(['Abhirup', 26, 'abhirup@example.com'])

```

7.4 Dictionary and Set Comprehensions

```

squares = {x: x**2 for x in range(5)}
# {0:0, 1:1, 2:4, 3:9, 4:16}

unique_lengths = {len(word) for word in ["hi", "hello", "hey"]}
# {2, 5, 3}

```

7.5 Choosing the Right Data Structure

- Use a **list** when order matters and you need to change items.
- Use a **tuple** for fixed, unchangeable groups of data.
- Use a **set** when you need uniqueness and fast membership checks, and order doesn't matter.
- Use a **dictionary** when you need to look things up by a key/label.

Chapter 8: Functions

8.1 Defining and Calling Functions

```
def greet(name):  
    """This function greets the person passed in as a parameter."""  
    return f"Hello, {name}!"  
  
message = greet("Abhirup")  
print(message)  # "Hello, Abhirup!"
```

Step by step:

- **def** starts a function definition, followed by the function name and parameters in parentheses.
- The string right after the definition is a **docstring**, documenting what the function does.
- **return** sends a value back to the caller; without it, a function returns **None**.

8.2 Default, Keyword, and Arbitrary Arguments

```
def power(base, exponent=2):  # default value for exponent  
    return base ** exponent  
  
print(power(3))              # 9, uses default exponent=2  
print(power(3, 3))           # 27, overrides default  
print(power(base=2, exponent=10))  # keyword arguments, order doesn't matter  
  
def add_all(*args):          # collects extra positional args into a tuple  
    return sum(args)  
  
print(add_all(1, 2, 3, 4))  # 10  
  
def describe(**kwargs):      # collects extra keyword args into a dict  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
describe(name="Abhirup", age=25)
```

8.3 Variable Scope

```
x = 10 # global variable

def modify():
    x = 20 # this creates a NEW local variable x, doesn't affect global
    print("Inside:", x)

modify()
print("Outside:", x) # still 10

def modify_global():
    global x
    x = 20 # this changes the actual global variable
modify_global()
print(x) # 20
```

8.4 Lambda Functions

Small, anonymous, single-expression functions.

```
square = lambda x: x ** 2
print(square(5)) # 25

add = lambda a, b: a + b
print(add(3, 4)) # 7

# Common use: as a key function for sorting
people = [("Alice", 30), ("Bob", 25), ("Eve", 35)]
people.sort(key=lambda person: person[1])
print(people) # sorted by age
```

LEARN WITH ABHIRUP

8.5 Recursion

A function that calls itself.

```
def factorial(n):
    if n <= 1: # base case, stops the recursion
        return 1
    return n * factorial(n - 1) # recursive case

print(factorial(5)) # 120
```

Step by step for **factorial(5)**:

1. **factorial(5)** calls **5 * factorial(4)**
2. **factorial(4)** calls **4 * factorial(3)**
3. This continues until **factorial(1)** returns **1**
4. The results multiply back up: $1*2*3*4*5 = 120$

8.6 Type Hints (Modern Python)

```
def add(a: int, b: int) -> int:  
    return a + b
```

Type hints document expected types; they are not enforced at runtime by default but help with readability and tools like **mypy**.



Chapter 9: Modules and Packages

9.1 Importing Modules

```
import math
print(math.sqrt(16))      # 4.0

from math import sqrt, pi
print(sqrt(25))           # 5.0
print(pi)                 # 3.14159...

import math as m          # aliasing
print(m.floor(4.7))       # 4

from math import *        # imports everything (avoid in large projects, can cause name
                           clashes)
```

9.2 Creating Your Own Module

Save this as `mymodule.py`:

```
# mymodule.py
def greet(name):
    return f"Hello, {name}"

PI = 3.14159
```

Then use it in another file in the same folder:

```
import mymodule
print(mymodule.greet("Abhirup"))
print(mymodule.PI)
```

9.3 Packages

A package is a folder containing an `__init__.py` file plus multiple modules, allowing you to organize related code.

```
myapp/
  __init__.py
  utils.py
  models.py
```

```
from myapp import utils
from myapp.models import User
```


9.4 The if `__name__ == "__main__"` Pattern

```
def main():  
    print("Running as a script")  
  
if __name__ == "__main__":  
    main()
```

This ensures `main()` only runs when the file is executed directly, not when it's imported as a module elsewhere.

9.5 pip and Virtual Environments

```
python -m venv myenv          # create an isolated environment  
source myenv/bin/activate     # activate on Linux/Mac  
myenv\Scripts\activate        # activate on Windows  
  
pip install requests          # install a package  
pip freeze > requirements.txt # save installed packages to a file  
pip install -r requirements.txt # install from that file
```

Virtual environments keep project dependencies isolated so different projects don't conflict.



Chapter 10: File Handling

10.1 Opening and Reading Files

```
file = open("data.txt", "r") # "r" = read mode
content = file.read()
file.close()

# Better: use "with" so the file closes automatically
with open("data.txt", "r") as file:
    content = file.read()
    print(content)

with open("data.txt", "r") as file:
    for line in file: # iterate line by line, memory efficient
        print(line.strip())

with open("data.txt", "r") as file:
    lines = file.readlines() # returns a list of all lines
```

10.2 Writing to Files

```
with open("output.txt", "w") as file: # "w" overwrites the file
    file.write("Hello, World!\n")
    file.writelines(["Line 1\n", "Line 2\n"])

with open("output.txt", "a") as file: # "a" appends without erasing existing content
    file.write("This gets added at the end\n")
```

10.3 File Modes Summary

- **"r"** — read (default, file must exist)
- **"w"** — write (creates file, overwrites if exists)
- **"a"** — append (creates file if it doesn't exist)
- **"r+"** — read and write
- **"b"** suffix (e.g. **"rb"**, **"wb"**) — binary mode, for images, PDFs, etc.

10.4 Working with CSV and JSON

```
import csv
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)      # each row is a list of strings

with open("data.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["name", "age"])
    writer.writerow(["Abhirup", 25])

import json
data = {"name": "Abhirup", "age": 25}
with open("data.json", "w") as file:
    json.dump(data, file)      # write dict as JSON

with open("data.json", "r") as file:
    loaded = json.load(file)   # read JSON back into a dict
    print(loaded["name"])
```

10.5 Working with Paths

```
from pathlib import Path

p = Path("folder/subfolder/file.txt")
print(p.exists())      # does it exist?
print(p.parent)        # "folder/subfolder"
print(p.name)          # "file.txt"
print(p.suffix)        # ".txt"

for file in Path(".").glob("*.py"): # list all .py files in current directory
    print(file)
```

Chapter 11: Exception Handling

11.1 try, except, else, finally

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print("This runs only if no exception occurred")
finally:
    print("This always runs, error or not")
```

Step by step:

- Code inside **try** runs first.
- If an error matching **ZeroDivisionError** occurs, the **except** block handles it instead of crashing the program.
- **else** runs only if no exception was raised.
- **finally** always runs, useful for cleanup like closing files or connections.

11.2 Catching Multiple Exceptions

```
try:
    value = int(input("Enter a number: "))
    result = 10 / value
except ValueError:
    print("That wasn't a valid number")
except ZeroDivisionError:
    print("Can't divide by zero")
except Exception as e:
    print(f"Unexpected error: {e}")
```

11.3 Raising Exceptions

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

try:
    set_age(-5)
except ValueError as e:
    print(e)  # "Age cannot be negative"
```

11.4 Custom Exceptions

```
class InsufficientFundsError(Exception):
    """Raised when a withdrawal exceeds the account balance."""
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(f"Cannot withdraw {amount}, balance is {balance}")
    return balance - amount

try:
    withdraw(100, 150)
except InsufficientFundsError as e:
    print(e)
```

11.5 Common Built-in Exceptions

- **ValueError** — invalid value for a valid type (e.g., `int("abc")`)
- **TypeError** — operation applied to the wrong type
- **IndexError** — list index out of range
- **KeyError** — dictionary key not found
- **FileNotFoundError** — file doesn't exist
- **ZeroDivisionError** — division by zero
- **AttributeError** — attribute or method doesn't exist on an object

LEARN WITH ABHIRUP

Chapter 12: Object-Oriented Programming (OOP) Basics

12.1 Classes and Objects

A class is a blueprint; an object (instance) is built from that blueprint.

```
class Dog:
    def __init__(self, name, breed):  # constructor, runs when object is created
        self.name = name              # instance attribute
        self.breed = breed

    def bark(self):                   # instance method
        return f"{self.name} says Woof!"

my_dog = Dog("Rex", "Labrador")      # creates an object (instance)
print(my_dog.name)                   # "Rex"
print(my_dog.bark())                 # "Rex says Woof!"
```

Step by step:

- `__init__` is called automatically when `Dog("Rex", "Labrador")` runs, setting up the object's initial state.
- `self` refers to the specific instance being created or used; it must be the first parameter of every instance method.

12.2 Class Attributes vs Instance Attributes

```
class Dog:
    species = "Canis familiaris"      # class attribute, shared by all instances

    def __init__(self, name):
        self.name = name              # instance attribute, unique to each object

d1 = Dog("Rex")
d2 = Dog("Buddy")
print(d1.species, d2.species)         # both "Canis familiaris"
print(d1.name, d2.name)               # "Rex" "Buddy"
```


12.3 Inheritance

```
class Animal:
    def __init__(self, name):
        self.name = name

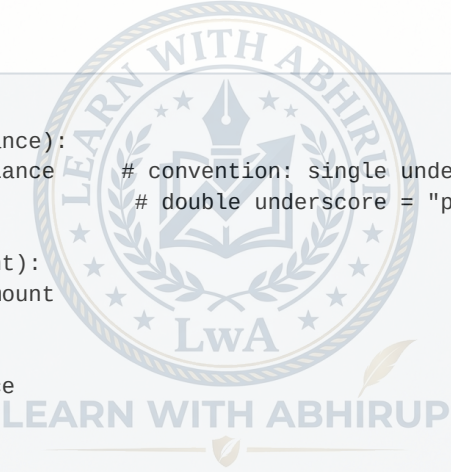
    def speak(self):
        return "Some generic sound"

class Cat(Animal):
    # Cat inherits from Animal
    def speak(self):
        # method overriding
        return f"{self.name} says Meow!"

class Puppy(Animal):
    def speak(self):
        return f"{self.name} says " + super().speak() # extends parent's method

c = Cat("Whiskers")
print(c.speak()) # "Whiskers says Meow!"
```

12.4 Encapsulation



```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance # convention: single underscore = "protected"
        self.__pin = "1234" # double underscore = "private" (name-mangled)

    def deposit(self, amount):
        self._balance += amount

    def get_balance(self):
        return self._balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance()) # 1500
# print(account.__pin) # AttributeError, name has been mangled to
# _BankAccount__pin
```

12.5 Polymorphism

Different classes can be used interchangeably if they share a common interface.

```
class Circle:
    def area(self):
        return "pi * r^2"

class Square:
    def area(self):
        return "side^2"

for shape in [Circle(), Square()]:
    print(shape.area()) # each object responds to area() in its own way
```

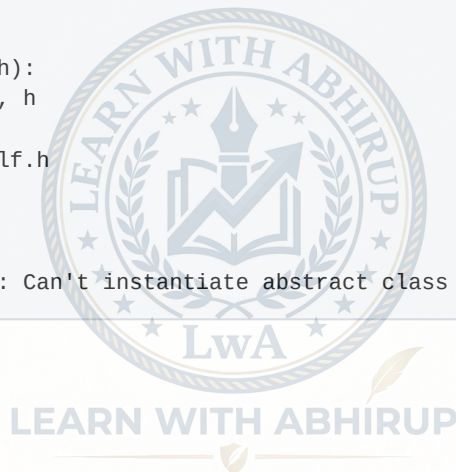
12.6 Abstraction

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass # forces subclasses to implement this method

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h
    def area(self):
        return self.w * self.h

r = Rectangle(4, 5)
print(r.area()) # 20
# s = Shape() # Error: Can't instantiate abstract class directly
```



Chapter 13: Advanced OOP

13.1 Magic (Dunder) Methods

Special methods surrounded by double underscores that customize built-in behavior.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __str__(self):
        # for print() and str()
        return f"Point({self.x}, {self.y})"

    def __repr__(self):
        # for debugging/representation
        return f"Point(x={self.x}, y={self.y})"

    def __add__(self, other):
        # enables the + operator
        return Point(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
        # enables ==
        return self.x == other.x and self.y == other.y

    def __len__(self):
        # enables len()
        return 2

p1 = Point(1, 2)
p2 = Point(3, 4)
print(p1 + p2)      # Point(4, 6), uses __add__
print(p1 == Point(1, 2)) # True, uses __eq__
```

LEARN WITH ABHIRUP

13.2 Properties (Getters and Setters the Pythonic Way)

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if value < 0:
            raise ValueError("Radius cannot be negative")
        self._radius = value

    @property
    def area(self):
        return 3.14159 * self._radius ** 2

c = Circle(5)
print(c.radius)      # 5, calls the getter
c.radius = 10        # calls the setter
print(c.area)        # computed automatically, 314.159
```

13.3 Class Methods and Static Methods

```
class Employee:
    company = "Learn with Abhirup"

    def __init__(self, name):
        self.name = name

    @classmethod
    def from_string(cls, data_string):
        name = data_string.split("-")[0]
        return cls(name)

    @staticmethod
    def is_valid_name(name):
        return isinstance(name, str) and len(name) > 0

e = Employee.from_string("Abhirup-25")
print(e.name)          # "Abhirup"
print(Employee.is_valid_name("Abhirup")) # True
```

13.4 Multiple Inheritance and MRO

```
class A:
    def greet(self):
        return "Hello from A"

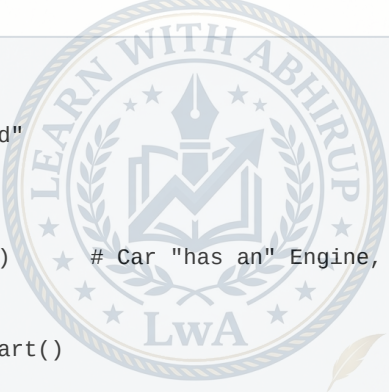
class B:
    def greet(self):
        return "Hello from B"

class C(A, B):
    pass

c = C()
print(c.greet())          # "Hello from A", follows Method Resolution Order (MRO)
print(C.__mro__)          # shows the lookup order: C -> A -> B -> object
```

13.5 Composition Over Inheritance

Instead of inheriting, classes can "have" other objects as attributes.



```
class Engine:
    def start(self):
        return "Engine started"

class Car:
    def __init__(self):
        self.engine = Engine()  # Car "has an" Engine, rather than "is an" Engine

    def start(self):
        return self.engine.start()

car = Car()
print(car.start())  # "Engine started"
```

Composition is often preferred over deep inheritance hierarchies because it keeps classes flexible and loosely coupled.

Chapter 14: Iterators and Generators

14.1 Iterables vs Iterators

An **iterable** is anything you can loop over (lists, strings, dicts). An **iterator** is an object that produces values one at a time using `__next__`.

```
nums = [1, 2, 3]
it = iter(nums)          # get an iterator from the list
print(next(it))          # 1
print(next(it))          # 2
print(next(it))          # 3
# print(next(it))        # raises StopIteration, no more items
```

14.2 Building a Custom Iterator

```
class Counter:
    def __init__(self, limit):
        self.limit = limit
        self.current = 0

    def __iter__(self):
        # makes the object iterable
        return self

    def __next__(self):
        # defines how to get the next value
        if self.current >= self.limit:
            raise StopIteration
        self.current += 1
        return self.current

for number in Counter(5):
    print(number)    # prints 1 2 3 4 5
```

14.3 Generators with yield

A generator is a simpler way to create an iterator using a function with `yield`.

```
def count_up_to(limit):
    current = 1
    while current <= limit:
        yield current    # pauses here, returns current, resumes on next call
        current += 1

for number in count_up_to(5):
    print(number)        # 1 2 3 4 5
```

Step by step:

- Calling `count_up_to(5)` doesn't run the function body immediately; it returns a generator object.
- Each call to `next()` (implicit in the `for` loop) runs the function until it hits `yield`, then pauses.
- State is preserved between calls, so `current` remembers its value.

14.4 Generator Expressions

Like list comprehensions but lazy (computed on demand), using parentheses instead of brackets.

```
squares = (x**2 for x in range(1000000)) # doesn't compute all million values upfront
print(next(squares))    # 0
print(next(squares))    # 1
# Memory efficient because values are generated one at a time, not all stored in memory
```

14.5 Why Generators Matter

Generators are memory efficient for large or infinite sequences because they generate values on demand instead of storing everything in memory at once.

```
def infinite_counter():
    n = 0
    while True:
        yield n
        n += 1

gen = infinite_counter()
for _ in range(5):
    print(next(gen)) # 0 1 2 3 4, could go forever if you kept calling next()
```

LEARN WITH ABHIRUP

Chapter 15: Decorators and Closures

15.1 Functions as First-Class Objects

In Python, functions can be passed around like any other value.

```
def shout(text):
    return text.upper()

def greet(func):
    return func("hello")

print(greet(shout))  # "HELLO", shout is passed as an argument
```

15.2 Closures

A closure is a function that remembers variables from its enclosing scope, even after that scope has finished executing.

```
def make_multiplier(factor):
    def multiplier(number):
        return number * factor
    return multiplier

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(5))  # 10
print(triple(5))  # 15
```

"factor" is remembered from the outer function

15.3 Writing a Decorator

A decorator is a function that wraps another function to extend its behavior without modifying its code.

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before the function runs")
        result = func(*args, **kwargs)
        print("After the function runs")
        return result
    return wrapper

@my_decorator
def say_hello(name):
    print(f"Hello, {name}!")

say_hello("Abhirup")
# Output:
# Before the function runs
# Hello, Abhirup!
# After the function runs
```

Step by step:

- `@my_decorator` above `say_hello` is shorthand for `say_hello = my_decorator(say_hello)`.
- Every call to `say_hello(...)` now actually calls `wrapper(...)`, which adds behavior before and after calling the real function.

15.4 Decorators with Arguments

```
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(3)
def greet():
    print("Hi!")

greet() # prints "Hi!" three times
```

15.5 Common Built-in Decorators

```
class MyClass:
    @staticmethod
    def utility():
        return "I don't need self"

    @classmethod
    def factory(cls):
        return cls()

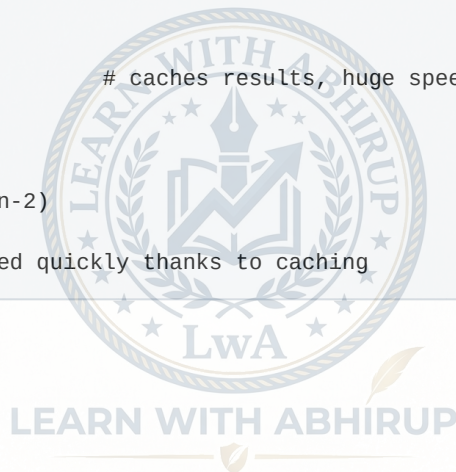
    @property
    def computed_value(self):
        return 42

from functools import wraps, lru_cache

def logged(func):
    @wraps(func)                # preserves original function's name/docstring
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@lru_cache(maxsize=None)        # caches results, huge speedup for expensive calls
def fib(n):
    if n < 2:
        return n
    return fib(n-1) + fib(n-2)

print(fib(30))    # computed quickly thanks to caching
```



Chapter 16: Context Managers

16.1 The with Statement

Used to manage resources like files or connections, guaranteeing cleanup even if an error occurs.

```
with open("file.txt", "r") as f:
    data = f.read()
# file is automatically closed here, even if an exception happened inside the block
```

16.2 Writing a Class-Based Context Manager

```
class ManagedFile:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode

    def __enter__(self):
        # runs when entering the "with" block
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_value, traceback):
        # runs when exiting
        self.file.close()
        return False # False means don't suppress exceptions

with ManagedFile("test.txt", "w") as f:
    f.write("Hello!")
```

16.3 Writing a Function-Based Context Manager

```
from contextlib import contextmanager

@contextmanager
def timer():
    import time
    start = time.time()
    yield # code inside "with" runs here
    end = time.time()
    print(f"Elapsed: {end - start:.4f} seconds")

with timer():
    total = sum(range(1000000))
```

Chapter 17: Regular Expressions

17.1 Basic Matching

```
import re

text = "My phone number is 123-456-7890"
match = re.search(r"\d{3}-\d{3}-\d{4}", text)
if match:
    print(match.group())    # "123-456-7890"
```

Common patterns:

- **\d** — any digit
- **\w** — any word character (letters, digits, underscore)
- **\s** — whitespace
- **.** — any character except newline
- ***** — zero or more, **+** — one or more, **?** — zero or one
- **^** — start of string, **\$** — end of string

17.2 Useful Functions

```
import re

re.findall(r"\d+", "a1 b22 c333")    # ['1', '22', '333'], all matches
re.sub(r"\d+", "#", "a1 b22 c333")   # "a# b# c#", replaces matches
re.split(r"\s*,\s*", "a, b, c")       # ['a', 'b', 'c'], split by pattern
re.match(r"Hello", "Hello World")    # matches only at the START of the string
re.fullmatch(r"\d+", "12345")        # matches the ENTIRE string
```

17.3 Groups

```
import re

text = "Name: Abhirup, Age: 25"
match = re.search(r"Name: (\w+), Age: (\d+)", text)
print(match.group(0))    # entire match: "Name: Abhirup, Age: 25"
print(match.group(1))    # first group: "Abhirup"
print(match.group(2))    # second group: "25"
```


Chapter 18: Functional Programming

18.1 map, filter, reduce

```
nums = [1, 2, 3, 4, 5]

squared = list(map(lambda x: x**2, nums))
# [1, 4, 9, 16, 25], applies function to every item

evens = list(filter(lambda x: x % 2 == 0, nums))
# [2, 4], keeps only items where the function returns True

from functools import reduce
total = reduce(lambda acc, x: acc + x, nums)
# 15, accumulates a single result by combining items pairwise
```

18.2 Comprehensions vs map/filter

Comprehensions are often more readable in Python:

```
squared = [x**2 for x in nums] # equivalent to map()
evens = [x for x in nums if x % 2 == 0] # equivalent to filter()
```

18.3 Pure Functions and Immutability

A pure function always returns the same output for the same input and has no side effects (doesn't modify external state).

```
def pure_add(a, b):          # pure: no side effects, deterministic
    return a + b

total = 0
def impure_add(a):          # impure: modifies external state
    global total
    total += a
```

Functional programming favors pure functions because they are easier to test, debug, and reason about.

Chapter 19: Multithreading, Multiprocessing, and Asyncio

19.1 Why Concurrency?

Concurrency lets a program do multiple things "at once" — useful when waiting on slow operations like network requests (I/O-bound) or when you want to use multiple CPU cores (CPU-bound).

19.2 Threading

```
import threading
import time

def worker(name):
    print(f"{name} starting")
    time.sleep(2)
    print(f"{name} finished")

t1 = threading.Thread(target=worker, args=("Thread-1",))
t2 = threading.Thread(target=worker, args=("Thread-2",))

t1.start()          # starts running in the background
t2.start()
t1.join()           # waits for t1 to finish before continuing
t2.join()
print("All threads done")
```

Note: Python's Global Interpreter Lock (GIL) means only one thread executes Python bytecode at a time, so threading helps with I/O-bound tasks (waiting on network/disk) but not CPU-bound tasks.

19.3 Multiprocessing

For CPU-bound work, use separate processes to bypass the GIL and use multiple cores.

```
from multiprocessing import Process

def compute_square(n):
    print(n * n)

if __name__ == "__main__":
    processes = [Process(target=compute_square, args=(i,)) for i in range(5)]
    for p in processes:
        p.start()
    for p in processes:
        p.join()
```

19.4 Asyncio

Used for high-performance I/O-bound concurrency using a single thread with an event loop.

```
import asyncio

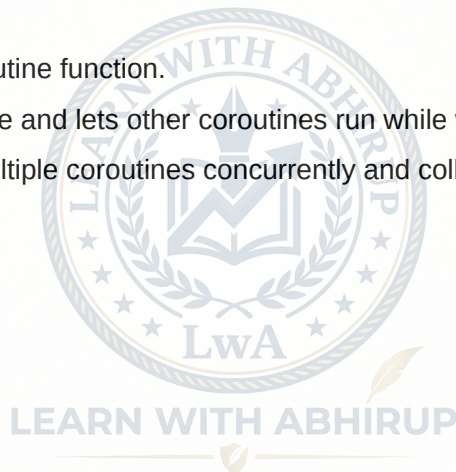
async def fetch_data(name, delay):
    print(f"{name} started")
    await asyncio.sleep(delay)      # non-blocking wait
    print(f"{name} finished")
    return f"{name} result"

async def main():
    results = await asyncio.gather(
        fetch_data("Task-1", 2),
        fetch_data("Task-2", 1),
    )
    print(results)

asyncio.run(main())
```

Step by step:

- **async def** defines a coroutine function.
- **await** pauses this coroutine and lets other coroutines run while waiting.
- **asyncio.gather** runs multiple coroutines concurrently and collects their results.



Chapter 20: Working with Dates and Time

20.1 The datetime Module

```
from datetime import datetime, timedelta

now = datetime.now()
print(now)                # current date and time
print(now.year, now.month, now.day)

specific = datetime(2024, 12, 25)
print(specific.strftime("%B %d, %Y")) # "December 25, 2024"

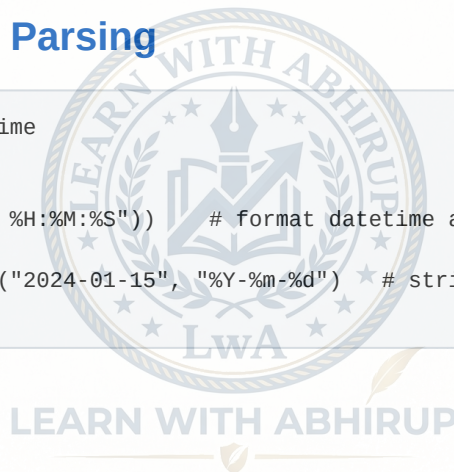
future = now + timedelta(days=7)      # 7 days from now
difference = future - now
print(difference.days)                # 7
```

20.2 Formatting and Parsing

```
from datetime import datetime

d = datetime.now()
print(d.strftime("%Y-%m-%d %H:%M:%S")) # format datetime as string

parsed = datetime.strptime("2024-01-15", "%Y-%m-%d") # string to datetime
print(parsed)
```



Chapter 21: Popular Third-Party Libraries (Overview)

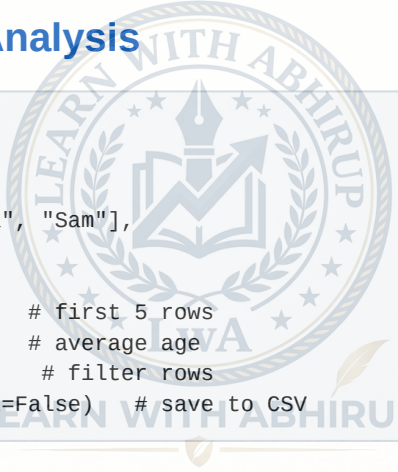
21.1 NumPy — Numerical Computing

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr * 2)           # [2, 4, 6, 8, 10], element-wise operations
print(arr.mean())        # average
print(arr.reshape(5, 1)) # reshape into a column

matrix = np.array([[1, 2], [3, 4]])
print(matrix.T)          # transpose
print(np.dot(matrix, matrix)) # matrix multiplication
```

21.2 Pandas — Data Analysis



```
import pandas as pd

df = pd.DataFrame({
    "name": ["Abhirup", "Riya", "Sam"],
    "age": [25, 22, 28]
})
print(df.head())          # first 5 rows
print(df["age"].mean())    # average age
print(df[df["age"] > 24])  # filter rows
df.to_csv("output.csv", index=False) # save to CSV
```

21.3 requests — HTTP Requests

```
import requests

response = requests.get("https://api.example.com/data")
print(response.status_code) # 200 means success
data = response.json()      # parse JSON response

response = requests.post("https://api.example.com/data", json={"key": "value"})
```

21.4 Others Worth Knowing

- **Matplotlib / Seaborn** — data visualization and charting
- **Flask / Django / FastAPI** — web frameworks
- **pytest** — testing framework
- **SQLAlchemy** — database ORM
- **BeautifulSoup / Scrapy** — web scraping
- **scikit-learn / PyTorch / TensorFlow** — machine learning and deep learning



Chapter 22: Testing in Python

22.1 Writing Tests with unittest

```
import unittest

def add(a, b):
    return a + b

class TestMath(unittest.TestCase):
    def test_add_positive(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative(self):
        self.assertEqual(add(-1, -1), -2)

if __name__ == "__main__":
    unittest.main()
```

22.2 Writing Tests with pytest (Simpler Syntax)

```
# test_math.py
def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0
```

Run with:

```
pytest test_math.py
```

22.3 Why Testing Matters

Automated tests catch bugs early, document expected behavior, and give confidence when refactoring or adding new features.

Chapter 23: Best Practices and PEP 8

23.1 Naming Conventions

- Variables and functions: `snake_case` (e.g., `total_price`)
- Classes: `PascalCase` (e.g., `BankAccount`)
- Constants: `UPPER_CASE` (e.g., `MAX_LIMIT`)
- Private attributes: prefix with underscore (e.g., `_internal`)

23.2 Code Style Tips

- Use 4 spaces per indentation level, never tabs.
- Keep lines under about 79-99 characters.
- Add blank lines to separate logical sections.
- Write clear docstrings for functions, classes, and modules.
- Prefer explicit, readable code over overly clever one-liners.

23.3 The Zen of Python

```
import this
```

This prints Python's guiding philosophy, including ideas like "Simple is better than complex" and "Readability counts."

LEARN WITH ABHIRUP

Chapter 24: Advanced Topics

24.1 The Global Interpreter Lock (GIL)

The GIL is a mutex that allows only one thread to execute Python bytecode at a time in CPython (the standard Python implementation). This simplifies memory management but limits true parallelism for CPU-bound multi-threaded code, which is why multiprocessing is used instead for CPU-heavy tasks.

24.2 Memory Management and Garbage Collection

Python manages memory automatically using reference counting plus a cyclic garbage collector for objects that reference each other in cycles.

```
import sys
a = [1, 2, 3]
print(sys.getrefcount(a))  # shows how many references point to this object

import gc
gc.collect()               # manually trigger garbage collection
```

24.3 Descriptors

Descriptors let you customize attribute access using `__get__`, `__set__`, and `__delete__`. Properties (seen earlier) are built on descriptors.

```
class PositiveNumber:
    def __set_name__(self, owner, name):
        self.name = "_" + name

    def __get__(self, instance, owner):
        return getattr(instance, self.name)

    def __set__(self, instance, value):
        if value < 0:
            raise ValueError("Must be positive")
        setattr(instance, self.name, value)

class Product:
    price = PositiveNumber()  # descriptor used as a class attribute
    def __init__(self, price):
        self.price = price

p = Product(100)
print(p.price)  # 100
# p.price = -5  # raises ValueError
```

24.4 Metaclasses

A metaclass is "the class of a class" — it controls how classes themselves are created. `type` is the default metaclass in Python.

```
class Meta(type):
    def __new__(mcs, name, bases, namespace):
        print(f"Creating class {name}")
        return super().__new__(mcs, name, bases, namespace)

class MyClass(metaclass=Meta):
    pass
# Output: "Creating class MyClass"
```

Metaclasses are an advanced feature mostly used in frameworks and libraries (e.g., Django's ORM) rather than everyday application code.

24.5 args unpacking and *, / in function signatures

```
def example(a, b, /, c, d, *, e, f):
    # a, b: positional-only (can't be passed as keywords)
    # c, d: normal (positional or keyword)
    # e, f: keyword-only (must be passed as keywords)
    return a + b + c + d + e + f

print(example(1, 2, 3, 4, e=5, f=6))
```

24.6 Walrus Operator (Python 3.8+)

```
data = [1, 2, 3, 4, 5]
if (n := len(data)) > 3:  # assigns AND checks in one expression
    print(f"List has {n} items, which is more than 3")
```

24.7 Match Statement (Python 3.10+)

Python's version of a switch/case statement, with powerful pattern matching.

```
def handle_command(command):
    match command.split():
        case ["go", direction]:
            return f"Moving {direction}"
        case ["look"]:
            return "Looking around"
        case ["take", *items]:
            return f"Taking {' '.join(items)}"
        case _:
            return "Unknown command"

print(handle_command("go north"))    # "Moving north"
print(handle_command("take sword shield")) # "Taking sword, shield"
```

24.8 Where to Go From Here

- Build small projects: a to-do list app, a calculator, a web scraper, a simple API.
- Read other people's code on GitHub to learn different styles and patterns.
- Learn a framework relevant to your interest: Django/Flask/FastAPI for web, pandas/NumPy for data, PyTorch/TensorFlow for machine learning.
- Practice regularly on platforms like LeetCode or HackerRank to sharpen problem-solving skills.
- Contribute to open-source projects to gain real-world experience.

Congratulations on completing this journey from the fundamentals of Python all the way to advanced, production-level concepts. Consistent practice is the key to mastery — keep building, keep experimenting, and keep learning.

— Learn with Abhirup

LEARN WITH ABHIRUP